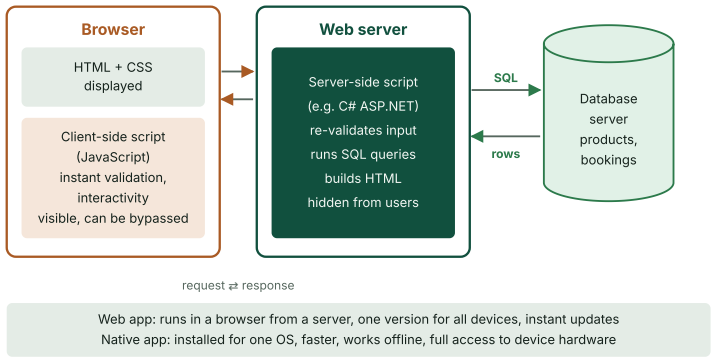
Web applications, intranets and web security

FD17 FACT FILE

Specification: students should be able to

- Compare client-side and server-side scripting and describe how dynamic web pages are generated using server-side scripts and databases
- Compare web applications with native applications
- Describe web-based services: e-services, e-government and portals
- Evaluate the use of an intranet by an organisation
- Evaluate methods of ensuring security over the internet: encryption (including public and private keys), HTTPS, Secure Sockets Layer (SSL/TLS), digital signatures and digital certificates

Key words: dynamic web page, client-side scripting, server-side scripting, SQL injection, web application, native application, e-service, e-government, portal, intranet, encryption, symmetric encryption, public key, private key, SSL/TLS, HTTPS, digital certificate, digital signature



A dynamic web page: client-side script runs in the browser, while the server-side script on the web server queries the database server and builds the HTML

Overview: applications and security on the Web

Many everyday tasks that once needed paper forms or a visit in person are now done through a web browser. Behind these services are web applications: programs that run partly in the user's browser and partly on a web server, and that build pages dynamically from information held in databases. Organisations use them to deliver e-services such as online shopping and banking, governments use them for e-government services such as licence renewals and tax returns, and portals collect many services together behind one consistent starting page. Inside an organisation, the same technology can form an intranet, a private web open only to staff. Because so much personal and financial data now travels across the public Internet, web applications must also be secure. Encryption makes intercepted data unreadable. Public and private key pairs allow strangers to communicate securely. TLS, the successor to SSL, protects the link between browser and server, and HTTPS is the result of carrying web traffic over it. Digital certificates prove that a public key belongs to the organisation it claims to, and digital signatures prove that a document is genuine and unaltered. This lesson covers how web applications are built and delivered, how organisations use them, and how they are kept secure.

Client-side and server-side scripting

A static page is a fixed HTML file that looks the same to every visitor until someone edits it. A dynamic page is generated when it is requested, usually by a server-side script that reads data from a database and builds HTML, so content can be personalised and kept current simply by changing the data. Client-side scripts, normally JavaScript, are downloaded with the page and run in the browser. They give an immediate response, for instance checking a form field as the user types or updating part of a page without reloading it, and they reduce the load on the server and the network. However, their code can be read, switched off or altered by the user, and their behaviour may vary between browsers. Server-side scripts, written in languages such as C# with ASP.NET, PHP or Python, run on the web server. They can query databases, process logins and payments and send emails, and because the code never leaves the server it remains hidden and secure. The cost is that every action needs a round trip to the server. Good design validates input on the client for usability and again on the server for security, using parameterised queries to prevent SQL injection attacks.

Web apps and native apps

A web application runs in a browser and is delivered from a server, so a single version works on any device and operating system with a modern browser. Nothing needs installing, updates take effect for every user the moment the server is changed, and the app can be shared simply by sending its URL. Its weaknesses are that it usually needs an internet connection, it may run more slowly, and it has more limited access to device hardware such as sensors, the file system or background notifications. A native application is written for a particular operating system, such as iOS or Android, and installed on the device. It runs faster, can use all of the device's features and works offline, and being listed in an app store helps people find it. However, separate versions must be developed and maintained for each platform, users must download updates, and app stores charge fees and impose approval rules. Progressive web apps narrow the gap by caching content for offline use and allowing installation to a home screen. The right choice depends on the purpose: a ticket booking service may suit a web app, while a fitness tracker that needs sensors and offline use suits a native app.

E-services, e-government and portals

E-services are services provided through electronic communication, usually the Internet. They include online shopping from internet-only firms and high street chains, online banking and insurance, live-chat helpdesks and digital editions of newspapers. For consumers the main benefit is convenience: services are available at any time from anywhere. Providers reach a wider customer base, gain an additional channel for contact with customers, can offer a larger range of products, and learn more about customers' preferences, which can give a competitive advantage; online-only retailers also avoid the cost of shop premises. E-government means governments providing their services online to the public, to businesses and to other bodies. It ranges from publishing information and guidance, through transactions such as vehicle tax or tax returns, to consultations and polls. It is generally faster and cheaper than paper-based systems, but governments must serve everyone, so people without equipment, connectivity or skills must not be disadvantaged. A portal is a website or page that gives consistent access to many connected services, often grouping links into rearrangeable blocks called portlets. An enterprise portal links an organisation's services for its staff or customers, usually after one log-in.

Evaluating an intranet

An intranet is a private network within an organisation, most often a private set of web pages and online services, that only authorised members can use. Its advantages are considerable. Policies, forms, training materials and staff directories can be published online, and because they are edited in one place only the latest version is available. Users sign in, so different groups can be shown only the links and documents relevant to them. Messaging, wikis, blogs and discussion boards improve communication and collaboration, online tools simplify tasks such as booking leave or scheduling meetings, and surveys help managers gauge and improve staff morale. There are also disadvantages. Setting up an intranet is expensive, and keeping its content accurate may require dedicated staff. Personal data on the intranet must be handled lawfully under the UK GDPR and the Data Protection Act 2018. A weak password could let an intruder reach confidential material, a risk that grows if staff can log in from outside the building. An organisation that relies heavily on its intranet can be badly disrupted if it goes down. A balanced evaluation weighs these points against the size and needs of the particular organisation and suggests ways of reducing the risks.

Encryption and public and private keys

Encryption converts readable plaintext into ciphertext using an algorithm and a key, so that anyone who intercepts the data cannot understand it. Only someone with the correct key can decrypt it. In symmetric encryption the same secret key is used to encrypt and to decrypt. This is fast and suits large amounts of data, but the sender and receiver must first share the key without anyone else learning it, which is difficult across the Internet. Public key, or asymmetric, encryption solves this by using a pair of mathematically linked keys, so that data encrypted with one key can only be decrypted with the other. The public key is published freely, while the private key is kept secret by its owner. To send someone a confidential message, you encrypt it with the recipient's public key; only the recipient's private key can decrypt it, so even if the message is intercepted it remains unreadable. Asymmetric encryption is much slower than symmetric encryption, so in practice the two are combined: public key techniques are used to agree a symmetric session key securely, and that session key then encrypts the actual data. This is exactly how TLS secures web traffic.

TLS, HTTPS, certificates and signatures

Secure Sockets Layer was the original protocol for creating an encrypted link between two devices; it has been replaced by Transport Layer Security, though the name SSL is still widely used. When a browser connects to a secure site, the two sides exchange greetings that list the encryption methods they support and include random values. The server sends its digital certificate, containing its public key. The browser checks the certificate, then the two agree a shared session key that an eavesdropper cannot work out, and all further data is encrypted with it. HTTP carried over this encrypted connection is HTTPS, shown by a padlock and using port 443. A digital certificate is issued and signed by a trusted certificate authority after checking the owner's identity. It contains the owner's name and domain, their public key, a serial number, the issuer and an expiry date, but never the private key. A digital signature serves a different purpose: it proves that a document came from a particular sender and has not been altered. The sender hashes the document and encrypts the hash with their private key. The receiver decrypts it with the sender's public key, recalculates the hash and compares the two values.

T Key terms from the mark schemes

Dynamic web page
A web page generated at the time of request, often from database data, so its content can change or be personalised.

Server-side scripting
Code that runs on the web server to process requests, access databases and generate pages before they are sent to the browser.

Native application
An application written for and installed on a specific operating system, with direct access to device hardware.
`Credited: Native app`

E-government
Government information and services provided electronically, usually online, to citizens, businesses and other organisations.

Intranet
A private network, usually of web pages and online services, within an organisation that only authorised members can access.

SSL/TLS
Protocols that create an authenticated, encrypted link between two devices, such as a browser and a web server; TLS is the modern successor to SSL.
`Credited: Secure Sockets Layer / Transport Layer Security`

Client-side scripting
Code, usually JavaScript, that is downloaded with a web page and run by the user's browser.

Web application
An application that runs in a web browser and is delivered from a server, processing user input and usually storing its data in a database on the server, without being installed on the device.
`Credited: Web app`

SQL injection
An attack in which SQL code is entered into an input field so that it is executed by the database.

Portal
A website or page that provides consistent access to a number of connected services from a single starting point.
`Credited: Enterprise portal`

Public key encryption
Encryption using a linked key pair: data encrypted with the freely available public key can only be decrypted with the owner's secret private key.
`Credited: Asymmetric encryption`

Digital certificate
An electronic document, issued and signed by a certificate authority, that links a public key to the identity of its owner.

Digital signature
A hash of a document encrypted with the sender's private key, used to verify who sent the document and that it has not been altered.

C C# example

Server-side script: building HTML from data (C#)

```
using System;
using System.Text;

(string Club, string Opponent, string Date)[] fixtures =
{
    ("Armagh", "Newry", "12 Apr"),
    ("Omagh", "Strabane", "19 Apr"),
    ("Armagh", "Lurgan", "26 Apr")
};
string club = "Armagh";              // from ?club=Armagh

var html = new StringBuilder("<ul>");
foreach (var f in fixtures)
    if (f.Club == club)
        html.Append($"<li>{f.Date}: v {f.Opponent}</li>");
html.Append("</ul>");
Console.WriteLine(html);
```

Exam focus

The specification asks you to evaluate the use of an intranet and to evaluate methods of ensuring security, so expect extended questions set in a business context. An evaluation needs advantages and disadvantages applied to the scenario and a justified conclusion, not two generic lists. For security questions, be precise about keys. A message for a recipient is encrypted with the recipient's public key and decrypted with their private key. A digital signature is created with the sender's private key and checked with the sender's public key. Mixing these up loses marks. Explain what each method achieves: encryption gives confidentiality, a signature gives authenticity and integrity, and a certificate links a public key to a verified owner. Do not say a certificate contains a private key. When describing HTTPS, mention both encryption of data in transit and authentication of the server by its certificate. For client-side and server-side comparisons, state where the code runs, who can see it, the effect on server load and response time, and why server-side validation is still needed. For web versus native apps, tie each point to the scenario, for example whether offline use or camera access is needed, and finish with a recommendation.

? Check yourself

1. Explain the difference between a static and a dynamic web page. [2]
2. Describe one advantage and one disadvantage of client-side scripting. [4]
3. Explain why form data should be validated on the server even if it has already been validated in the browser. [2]

Answers: 1. A static page is a fixed HTML file that is the same for every visitor until edited (1) / a dynamic page is generated when requested, e.g. by a server-side script using database data, so content can change or be personalised (1) 2. Advantage (2), one from: runs in the browser so gives an immediate response (1) e.g. instant validation feedback (1) / reduces load on the server/network (1) because fewer requests are sent (1). Disadvantage (2), one from: code is visible/can be altered or disabled by the user (1) so cannot be relied on for security/validation (1) / behaviour may differ between browsers (1) so must be tested on many (1) 3. Client-side validation can be bypassed, e.g. by disabling JavaScript or sending a request directly (1) / so the server must check data to protect the database, e.g. against SQL injection or invalid data (1)